Carla Aranda 100523031 - Marina Juzgado 100523023

PHASE 1: Selection of the state information and reward function

1. Select the attributes to be used to represent a game state.

To choose the attributes, we will take into account a rectangular (square in this case) board divided in the following way: four quadrants of equal size, bounded by two lines crossing the middle of the board vertically and horizontally.

As it was required the information to be generic and not too many attributes so the Q-table is not excessively big, as well as it was recommended to choose nominal attributes, the selected attributes are the following:

- relative_pos: relative position of the food (0, 1, 2, 3) depending on the position of the food respect to the snake.
- danger_right: if it is save to turn right (TRUE/FALSE).
- danger_left: if it is save to turn left (TRUE/FALSE).
- danger_up: if it is save to turn up (TRUE/FALSE).
- danger_down: if it is save to turn down (TRUE/FALSE).
- current_dir: direction at that moment.

In the class SnakeGameEnv of the document snake_env.py, we updated the get_state function with our own attributes.

```
def get_state(self):
 head_x, head_y = self.snake # Snake head position
 food_x, food_y = self.food # Food position

 # --- Relative position ---
 if self.food_pos[0] <= self.snake_pos[0] and self.food_pos[1] <= self.snake_pos[1]:
 relative_pos = "0"
 if self.food_pos[0] <= self.snake_pos[0] and self.food_pos[1] > self.snake_pos[1]:
 relative_pos = "2"
 if self.food_pos[0] > self.snake_pos[0] and self.food_pos[1] <= self.snake_pos[1]:
 relative_pos = "1"
 if self.food_pos[0] > self.snake_pos[0] and self.food_pos[1] > self.snake_pos[1]:
 relative_pos = "3"

 # --- Danger attributes ---
 #Predict next positions for the four directions
 left_pos = [head_x - 10, head_y]
 right_pos = [head_x + 10, head_y]
 up_pos = [head_x, head_y - 10]
 down_pos = [head_x, head_y + 10]

 #Check if there are walls or if there is the snake body
 wall_left = (left_pos[0] < 0)
 wall_right = (right_pos[0] >= self.frame_size_x)
 #Use frame_size as in the statement it is said it must play in any board
 wall_up = (up_pos[1] < 0)
 wall_down = (down_pos[1] >= self.frame_size_y)

 body_left = left_pos in self.snake_body
 body_right = right_pos in self.snake_body
 body_up = up_pos in self.snake_body
 body_down = down_pos in self.snake_body

 #Define dangers
 danger_left = wall_left or body_left
 danger_right = wall_right or body_right
 danger_up = wall_up or body_up
 danger_down = wall_down or body_down

 #Current direction
 current_dir = self.direction

 #Return state
 state = (relative_pos, danger_right, danger_left, danger_up, danger_down, current_dir)

 return state
```

2. The tuples state and next state has the same attributes, but representing different ticks.

We need to update the function step with the attributes that we have created, we just added a couple of lines of code as well as changed the order to first get the initial state, then update with action to then add the next state to the initial one after changing food position and reward. In this way, it would be returned the state, next_state, reward and game over.

3. Design a reward function that allows the snake to maximize the score of a game.

The reward could be either positive or negative. We set the reward as positive when the snake eats food, and alternatively, we set the reward as negative when it collides with a wall or itself and small negative rewards when it moves but does not eat food.

```
def calculate_reward(self):
 reward = 0
 # 1. Check if snake eats the food
 if self.snake_pos[0] == self.food_pos[0] and self.snake_pos[1] == self.food_pos[1]:
 reward += 100 # Positive reward for eating food
 # 2. Check if snake collides
 if self.check_game_over(): # As when it collides the game is over
 reward -= 300 # Large negative reward for game over
 # 3. Small negative reward for each move to encourage efficient movement
 reward -= 1

 return reward
```

PHASE 2: Generation of the agent

1. Implement the code of the missing functions.

As we also did in the previous phase, we calculate an optimal function of rewards that encourages the snake to maximize its score. In the same way, we updated the function `get_state` with the new attributes that we created and summarize the game.

Furthermore, in the QLearning class we added every required tuple indicated in the statement, for which we need to know if a state is terminal or not. We added a line in the step() function that defines `next_state` as `None` in case it is terminal.

```
def update_q_table(self, state, action, reward, next_state, agent):
 # Convert state to index
 state_idx = agent.encode_state(state)

 # Read the current q-value for (state, action)
 predict = self.q_table[state_idx, action]

 # Handle terminal state safely
 if next_state is None:
 target = reward # Terminal state
 else:
 next_state_idx = agent.encode_state(next_state)
 target = reward + self.gamma * np.max(self.q_table[next_state_idx])

 # Q-learning update
 self.q_table[state_idx, action] = (1 - self.alpha) * predict + self.alpha * target
```

To update SnakeGame.py, we notice that there are missing two values: `number_states` and `num_episodes`. To add them, we do the following:

- For `number_states` we contemplate all possible states there could be depending on our attributes of step, multiplying every possibility to generate the number in the following way:

relative_pos	0, 1, 2, 3	4
danger_right	TRUE, FALSE	2
danger_left	TRUE, FALSE	2
danger_up	TRUE, FALSE	2
danger_down	TRUE, FALSE	2
current_dir	UP, DOWN, LEFT, RIGHT	4

  `number_states` = 4×2×2×2×2×4 = 256

- In `num_episodes`, we just set a number of episodes for training. For a decent train, we choose 5000.

We also did several changes to the main loop as code was required in the statement. First, we added a couple of lines to choose the best action for the state and algorithm, then, update the q-table, `state` and `total_reward`:

```
while not game_over:
 action = ql.choose_action(state)
 next_state, reward, game_over = env.step(action)
 # Choose the best action for the state and possible actions from the q_learning algorithm
 # Call the environment step with that action and get next_state, reward and game_over variables
 if training:
 ql.update_q_table(state, action, reward, next_state)

 # Update the state and the total_reward.
 state = next_state
 total_reward += reward
```

2. Implement the agent functionality of the tutorial 4 in this new code so that a Q-table is built and updated according to the Q-Learning algorithm.

For this, we created a new class similar to the one performed in Tutorial 4 in order to program an agent according to the q-learning algorithm.

```
class SnakeAgent:
 def __init__(self, n_states, n_actions, alpha=0.1, epsilon=0.1, discount=0.9, q_table_file="q_table.txt"):
 self.n_states = n_states
 self.n_actions = n_actions
 self.alpha = alpha # Learning rate
 self.epsilon = epsilon # Exploration probability
 self.discount = discount # Discount factor
 self.q_table_file = q_table_file

 try: #In case q-table file exists, load it and if not create it
 self.q_table = np.loadtxt(self.q_table_file)
 if self.q_table.shape != (n_states, n_actions):
 raise ValueError("Invalid Q-table shape, reinitializing.")
 except:
 self.q_table = np.zeros((n_states, n_actions))

 def save_q_table(self): #Save to a txt file
 np.savetxt(self.q_table_file, self.q_table)

 def encode_state(self, state):
 #Encode into a single integer among the possible states (256 possible, from 0 to 255)
 relative_pos, danger_right, danger_left, danger_up, danger_down, current_dir = state

 # Map the relative_pos
 rel = int(relative_pos) # already a string "0", "1", "2", "3", so int() is ok

 # Map the dangers: True = 1, False = 0
 dr = int(danger_right)
 dl = int(danger_left)
 du = int(danger_up)
 dd = int(danger_down)

 # Map the current direction
 dir_map = {"UP": 0, "DOWN": 1, "LEFT": 2, "RIGHT": 3}
 cd = dir_map[current_dir]

 # Combine all into a unique number
 index = rel
 index = index * 2 + dr
 index = index * 2 + dl
 index = index * 2 + du
 index = index * 2 + dd
 index = index * 4 + cd

 return index

 def get_q_value(self, state, action): #Get state and action and find the q-value in the table
 state_idx = self.encode_state(state)
 return self.q_table[state_idx, action]

 def get_action(self, state): #decide wwhat action it is better to take now
 if np.random.uniform(0, 1) < self.epsilon:
 return random.randint(0, self.n_actions - 1) # Random action with p epsilon
 else:
 state_idx = self.encode_state(state)
 return np.argmax(self.q_table[state_idx]) # Choose best action

 def update(self, state, action, next_state, reward): #Update q-table
 state_idx = self.encode_state(state)
 next_state_idx = self.encode_state(next_state)

 predict = self.q_table[state_idx, action] #current q-value of the state
 target = reward + self.discount * np.max(self.q_table[next_state_idx]) #q-learning rule
```

After doing this and making the necessary changes in the main class to be able to implement the agent, we run the code. Initially, the snake immediately goes up and crushes with the wall. This beahivor is due to the parameters in the q-learning class have not been adjusted.

In this way, after thinking about the best values for the game taking into account that this is its first attempt and a deeper exploration will be needed in the future, we change them to the following:

alpha	gamma	epsilon	epsilon_min	epsilon_decay
0.3	0.9	1	0.1	0.9995

We chose these parameters to improve the snake performance initially, so these values are just useful for the first few attempts, specially choosing a very high epsilon so the agent explores instead of using previous examples, as now we do not have other examples. These parameters will be used for the first 3000 episodes.

We can see that as the episodes are loading, the agent performance improves due to the total rewards for the first agents were negative and as it trains more episodes the rewards started to get slightly better. However, even then it continues giving negative scores, indicating that the values of te parameters are not the best as it explores too much and looses score since it is not eating the food.

After 3000 episodes we obtain a q-table with practically all the values set to 0 except some of them. This is due to we have 256 different states but not all of them are reached in every game, so there would clearly be unexplored ones.

Now, we will check how the agent works when we set alpha and epsilon to 0. Watching the performance of the agent, we see that it performs correctly and eats the fruits without crushing, although sometimes it gets stuck at some point. To avoid this, we will set the epsilon parameter to 0.05, to allow a bit of exploration just in some necessary cases. As it still performs with some errors, we decided to train it more with the initial parameters increasing the number of episodes.

After the new training and some tests setting the parameters to 0 we concluded that the agent has a good enough training. To be sure that it has a good performance, we tested it in a bigger board (400x400) and checked that it does perform good in other sizes of boards. As it worked perfectly we could proceed with the next step

3. Select the best value for the α, ϵ and γ attributes

The best values for the attributes are the ones that balance exploration, learning and execution so that an efficient and intelligent agent would play the game.

First, we configured the attributes to extend the training but with other characteristics trying to implement that the agent would still learn from this data. This means it is not the final test. Moreover, we trained the agent this time with a wider board. We chose the following attributes for the next attempt:

alpha	gamma	epsilon	epsilon_min	epsilon_decay
0.1	0.95	0.5	0.01	0.999

With these parameters we wanted to add 1500 more episodes. However, it took an extremely long time as since episode 200 more or less the agent started to performance extremely well, with total rewards of an average of 20000 or more. Nevertheless, sometimes it still obtains very bad negative results as it enters into a loop to avoid collapsing instead of looking for fruits.

Then, we tried a third round of possible attributes parameters. This one has been the one that worked the best for training, obtaining always positive results which means it always finds food more than one time. Because of this, we state that the best parameters after a proper training are the following:

alpha	gamma	epsilon	epsilon_min	epsilon_decay
0.1	0.99	0.1	0.01	0.995

4. The operation of the agent is as follows the statements.

As we had showed in the processing and creation of the functions, it does follow the statement. This means that when running the code, it first reads the q-table in case there is one, if not just creates it (done in the q_learning document). Second, in the main class we call a method to create a tuple with the necessary attributes (`state, action, reward` and `next_state`), these tuples also update the q-table thanks to a method in q_learning.

To continue with, we use a ε-greedy strategy specifically in the `choose_action` function of the q_learning document in the following way:

```
while not game_over:
 action = ql.choose_action(state)
 next_state, reward, game_over = env.step(action)
 # Choose the best action for the state and possible actions from the q_learning algorithm
 # Call the environment step with that action and get next_state, reward and game_over variables
 if training:
 ql.update_q_table(state, action, reward, next_state)

 # Update the state and the total_reward.
 state = next_state
 total_reward += reward
```

To finish with, at the end of each episode we save the q-table thanks to a simple function in the main class so the next episode also has the information of the previous one to be able to learn.

PHASE 3: Enhancement of the agent

1. Once it works with any board size, change the state so it takes into account the snake's body. Consider different alternatives balancing benefits and q table size.

We have tested the attributes and parameters for every board size with the optimal epsilon and alpha that we stated before. It works well both in small boards like the initial one (150x150) and in bigger ones like the last one (480x480). The only exception are extremely small board sizes (50x50), that give problems as the snake is not able to fit in them, making python crash and the q-table to be deleted.

Now, to take into account the snake's growing body, we turn it into `True` the `growing_body` parameter. As we englobed the snake body in the state with the danger attributes that take into account both the body and the limits of the board, we do not need to do any important changes as we already have information about the body in the state.

After some tests and seeing how the snake enters into a loop with itself until it crashes, we also added four more attributes that indicate for all four directions in the case that you continue straight until the wall if it would crash with its own body. we added it to the `get_state` function in the snake_env document:

```
body_in_sight_up=any(
 [head_x, y] in self.snake_body for y in range(head_y - 10, -10, -10))
 body_in_sight_down=any(
 [head_x, y] in self.snake_body for y in range(head_y + 10, self.frame_size_y, 10))
 body_in_sight_left=any(
 [x, head_y] in self.snake_body for x in range(head_x - 10, -10, -10))
 body_in_sight_right=any(
 [x, head_y] in self.snake_body for x in range(head_x + 10, self.frame_size_x, 10))
```

The advantages are that we now have much more useful information, the disadvantages that the q-table went from 256 states to 4096 states. This means that we would need approximately ten times that number of episodes, which is 40.000, a very huge number of states taking into account that each game is one state. The problem is that we can not use that many states as python would not run the q-table. Because of this problem, we finally decided to delete the new attributes as they were the less important because they are just useful for long snakes.

2. Train the model and evaluate your agent on as many runs as possible using different board sizes configurations.

We decided to train the snake with a slight modification of the optimal parameters so it does explore a bit more by increasing epsilon as it already knows how to search the food thanks to the phase two so now it just has to learn the way to avoid its own body. We ran 500 episodes with the following attributes and setting the `Training` into `True` again, first using the 150x150 board and then making it gradually bigger, for each board size making 500 episodes.

alpha	gamma	epsilon	epsilon_min	epsilon_decay
0.3	0.99	0.5	0.05	0.995

With those parameters the agent performed well sometimes but no exactly excellent. To achieve better results, we selected the best attributes that we chose on the previous phase to see how it does perform now repeating the process of the 500 states for each board size.

It does not perform really well, this means that lower epsilon and alpha performs worse, which at the same time means it needs to explore more. We did not see better improvement, so we decided to create a new q-table as the other one had too many instances without taking into account that the snake could grow, so danger was much less frequent. Again, after performing all the procedure once more, we tried with the optimal attributes. Now, since the beginning the performance is better, obtaining positive total rewards with just some occasional exceptions.

When we had enough training in the q-table, we set the alpha and epsilon to 0 and test it with different board sizes. We tried different board sizes from 150 to 450 seeing the agent performing well with positive total rewards in every board.

3. Report both quantitative and qualitative results on the agents' behavior.

First, to report the quantitative results on the agent's behaviour we added a bit of extra code to the main class so there are reports of average total rewards for each 50 episodes apart from the particular information in each episode. This is the code we used to add the information:

```
scores = []
 total_score_window = 0

scores.append(total_reward)
 total_score_window += total_reward

 if (episode + 1) % 50 == 0:
 avg_100 = total_score_window / 50
 max_100 = max(scores[-50:])
 min_100 = min(scores[-50:])
 print(f"--- Episode {episode - 48}-{episode + 1} ---")
 print(f"Average reward: {avg_100:.2f}")
 print(f"Max reward: {max_100}")
 print(f"Mín reward: {min_100}")
 print("-----------------------------")
 total_score_window = 0
```

The results are the following for every first 50 episodes in each board size in the testing mode:

Board 150x150: - Average reward: 1091.88 - Maximum reward: 2271 - Minimum reward: -42	Board 250x250: - Average reward: 1776.40 - Maximum reward: 3960 - Minimum reward: 135
Board 350x350: - Average reward: 1876.90 - Maximum reward: 4110 - Minimum reward: 157	Board 450x450: - Average reward: 2009.18 - Maximum reward: 4226 - Minimum reward: 111

With this information we can conclude with several qualitative results:

- The agent slightly performs better in bigger boards, this is perhaps because there is more space for the body to grow without crashing with itself so it definitely adapts to different board sizes.
- In every game the snake has eaten food as the penalization for crashing is -300 and the reward for eating a fruit is +100. This means that even in the minimum scores it has eaten food.
- This time we have seen that the snake is not in infinite loops but actually plays the game properly, mostly crashing when it is impossible to avoid it
- It prioritizes fast paths except when it has to avoid its own body that first avoids it and then chooses the fastest path to obtain the reward. At the same time, sometimes to avoid its own body it enters in a loop around itself that finishes in an inevitable crash.

Difficulties and conclusions

Most of the conclusions of this second assignment are related with the good performance of the Q-learning to develop an agent that plays the Snake game. Compared to the one created in the first assignment, this new agent performs much better than the first one, playing in a more natural and effective way. This could mainly be because of the reward system of the Q-learning method that motivates the agent to play better, knowing what actions give a reward and which action do not. In the agent of the first assignment there were not rewards, it just tried to copy the way a human played which is not the best method as a human does not necessarily plays in the best and more effective way.

In the same way, this project improved us to think about useful attributes for the agent such as the relative position between the food and the snake among others. With this method it was necessary to think about understandable but not too computationally expensive attributes to have a manageable q-table but with enough information for a good game play.

On the other hand, there have been some difficulties along this project. Mainly, the time it takes to gather all the episodes and information for the q-table even in very high speeds of the game as for example, in our case, for 256 different states we initially did 5000 different episodes that really took time. There have not been further difficulties more than the time spent in the assignment as the Python errors and establishment of the attributes alpha and epsilon were easily solved.